

■■ SIH 2026: AegisBorder AI — Strict Judge Question Bank

Comprehensive Technical Defense, Code Cross-Examination & Vulnerability Audit

Problem Statement: SIH26188 • **Team:** The Deciders • **Project:** AegisBorder AI • **Repo:**

<https://github.com/Jishnu09-siuu/AegisBorder-AI.git> • **Live Demo:** <https://rakshak-ai-omega-wheat.vercel.app/>

■■ **PURPOSE OF THIS DOCUMENT:** This manual simulates the exact inquiries, cross-examinations, and architectural stress-tests that a senior SIH technical jury will impose on your team. It exposes discrepancies between pitch claims and source code reality, providing the exact questions and judge trap-points across all 17 defense dimensions.

1. Codebase Reality vs. Marketing Claim Verification Audit

Stated Pitch Claim	Source Code Implementation	Verdict	Judge Cross-Examination Trap
Deep Learning 128-d Face Vectors	compute_face_feature_vector in face_verifier.py computes spatial 8x8 intensity histograms (1024) + Sobel gradient magnitudes (256).	Heuristic Only	'Which neural net produced your 128-d embeddings? You aren't using FaceNet or ArcFace; you are using a classical Sobel/intensity histogram.'
Live Interpol Red Notice Query	watchlist_db.py has a static in-memory dictionary KNOWN_WATCHLIST with 3 seeded records.	Simulated	'Did Interpol actually issue you an API token? What is your failover when the external government network drops?'
Bayesian Risk Scoring Engine	risk_engine.py uses linear weighted sums (30-25-25-20) with hardcoded heuristic risk floors.	Rule-Based	'Where are the prior probabilities and conditional likelihood distributions? This is a weighted average, not Bayes rule.'
ICAO 9303 7-3-1 Checksum Math	mrz_parser.py implements the full mod10 7-3-1 algorithm for TD1, TD2, and TD3 specifications.	Fully Verified	'If a criminal manufactures a fake passport with mathematically correct checksums, does your MRZ module catch it?'
Error Level Analysis (ELA)	ela.py re-saves image at 90% JPEG quality, subtracts difference, and scales deltas.	Fully Verified	'What happens if the forged passport was uniformly re-compressed as a whole? Doesn't ELA fail completely?'
Sub-1.2 Second Screening Latency	Measured on local fast multi-core machine without network round-trip overhead.	Conditional	'On Vercel serverless with a cold start, your Python function takes 3–6 seconds. How did you benchmark 1.2s?'

2. Complete 17-Round Technical Examination Question Bank

ROUND 1: PROJECT FUNDAMENTALS & ELEVATOR PITCH

Q1.1: Explain AegisBorder AI in exactly 30 seconds as if you are presenting to the national jury.

• *Judge Focus: Conciseness, eliminating fluff, stating the exact input, throughput, and output.*

Q1.2: What exact problem are you solving that commercial e-Gates (like SITA or Vision-Box) haven't already solved?

• *Judge Focus: Identifying the exact technical differentiator without sounding generic.*

Q1.3: Who is the primary operational user of this dashboard: the passenger, the border guard, or immigration intelligence?

• *Judge Focus: Assessing operational workflow realism and human-in-the-loop design.*

Q1.4: What is the exact mathematical input to your system and what is the exact output structure?

• *Judge Focus: Verifying data contract understanding (Base64 JPEG + JSON metadata -> 0-100 score + JSON audit).*

ROUND 2: TECHNICAL ARCHITECTURE & END-TO-END FLOW

Q2.1: Walk me step-by-step through what happens under the hood from the millisecond a passport is scanned to when the risk gauge renders.

• *Judge Focus: Traceability from React state -> Vite proxy -> FastAPI endpoint -> OpenCV pipeline -> response parsing.*

Q2.2: In which file and function does the FastAPI backend receive the uploaded document image?

- Judge Focus: Direct code knowledge: ``backend/main.py` -> @app.post('/api/screen-document') -> screen_document().`

Q2.3: How are you avoiding memory leaks in OpenCV when processing high-resolution passenger scans concurrently?

- Judge Focus: Handling numpy memory allocation, garbage collection, and PIL image buffer release.

Q2.4: Your frontend runs on Vercel Edge and proxies to a Python serverless function via `vercel.json`. What happens when Vercel hits its 1024MB memory limit during SciPy edge transforms?

- Judge Focus: Understanding production deployment constraints and serverless memory ceiling.

ROUND 3: ICAO DOC 9303 & MRZ CHECKSUM MATHEMATICS

Q3.1: Explain the 7-3-1 weighting algorithm mathematically. Why were 7, 3, and 1 chosen by ICAO instead of sequential weights?

- Judge Focus: Cryptographic and mathematical properties: coprime weights with base 10 minimize transposition error propagation.

Q3.2: Calculate the check digit for document number 'A209841' right now using pen and paper.

- Judge Focus: Testing raw mathematical competence on the spot without relying on code.

Q3.3: If a document passes all 4 ICAO check digits with 100% accuracy, does that prove the passport is genuine?

- Judge Focus: Critical thinking: Checksum only proves internal mathematical consistency. A counterfeiter can easily generate valid checksums for a completely synthetic identity.

Q3.4: How does your MRZ parser handle TD1 (ID cards) vs TD2 (Visas) vs TD3 (Passports)? Where is that auto-detection implemented?

- Judge Focus: Code verification in ``backend/parsers/mrz_parser.py``: line count (2 vs 3) and character length (44, 36, 30).

ROUND 4: MULTI-SPECTRAL DIGITAL FORENSICS (ELA, NOISE, SOBEL)

Q4.1: Explain the exact physics and mathematical basis of Error Level Analysis. Why does resaving at 90% quality reveal tampering?

- Judge Focus: Understanding JPEG discrete cosine transform (DCT) quantization tables and compression error delta accumulation.

Q4.2: What is the primary vulnerability of ELA? If a forger saves their modified passport and then flattens and re-compresses the entire image once, what does ELA output?

- Judge Focus: Identifying false negatives: uniform re-compression resets the error surface, rendering ELA blind.

Q4.3: Why did you combine ELA with Laplacian Noise Variance and Sobel Edge Gradients?

- Judge Focus: Defense-in-depth: Laplacian catches blur/smoothing tools, while Sobel catches photo border boundary jumps.

Q4.4: Show me the exact mathematical formula you use in ``backend/forensics/ela.py`` to calculate the tamper score from the difference image.

- Judge Focus: Code accuracy: ``np.mean(diff_np)``, ``np.percentile(diff_np, 95)``, and scaling thresholds.

ROUND 5: BIOMETRICS & FACIAL VERIFICATION

Q5.1: In your documentation, you claim '128-dimensional facial vectors'. Open ``backend/biometrics/face_verifier.py``. Are you using FaceNet, ArcFace, dlib, or classical histograms?

- Judge Focus: Catching over-claiming: The code actually uses 8x8 block histograms (1024) + Sobel gradient magnitude (256).

Q5.2: Why did you choose Cosine Similarity instead of Euclidean (L2) distance for comparing facial vectors?

- Judge Focus: Directional orientation in high-dimensional normalized space vs magnitude sensitivity under uneven illumination.

Q5.3: You set your match threshold at 0.75. How was this number determined? Did you generate an ROC curve on LFW or FERET benchmarks?

- Judge Focus: Empirical validation vs arbitrary guessing of security-critical thresholds.

Q5.4: How does your Presentation Attack Detection (PAD) differentiate between a live human face and a high-resolution iPad screen displaying a photo?

- Judge Focus: Understanding 2D Fast Fourier Transform (FFT) frequency spectrum rings for screen moiré patterns in ``check_liveness_and_anti_spoofing()``.

ROUND 6: WATCHLIST & INTERPOL INTEGRATION

Q6.1: Your video states you query Interpol Red Notices. Are you actually connected to Interpol's 24/7 I-Link or SLTD database?

- Judge Focus: Absolute honesty: Admitting it is an in-memory simulated database (`KNOWN_WATCHLIST` in ``watchlist_db.py``) designed to mimic law enforcement protocols.

Q6.2: How do you handle name transliteration and typos (e.g., 'Mohammed' vs 'Mohamed') in your watchlist screening?

• *Judge Focus: Fuzzy string matching algorithms (Levenshtein distance, Jaro-Winkler, Soundex/Metaphone) vs exact string matching.*

Q6.3: What privacy and security safeguards prevent a corrupt border operator from dumping your entire watchlist database?

• *Judge Focus: Data encryption at rest, role-based access control (RBAC), and blind index hashing.*

ROUND 7: COMPOSITE RISK ENGINE & DECISION TIERS

Q7.1: Your weights are MRZ=30%, Forensics=25%, Biometrics=25%, Watchlist=20%. Why is Watchlist only 20% if a criminal hit is a critical event?

• *Judge Focus: Understanding hard overrides vs linear weights: Explain the 'Risk Floor' mechanism where a watchlist hit automatically forces risk $\geq 92\%$ regardless of weights.*

Q7.2: Your project claims 'Bayesian Risk Scoring'. Where are the Bayesian prior probabilities, evidence updates, and conditional likelihood matrices in `risk\_engine.py`?

• *Judge Focus: Exposing terminology inflation: Defend whether the system uses true Bayesian inference or heuristic multi-factor weighted aggregation.*

Q7.3: What exact criteria separate 'Secondary Inspection' (Counter 4B) from 'Immediate Detention'?

• *Judge Focus: Operational protocol justification: Document anomalies vs verified identity fraud / criminal alerts.*

ROUND 8: SECURITY, ADVERSARIAL ATTACKS & API DEFENSE

Q8.1: If I submit a 50MB maliciously crafted TIFF file to `/api/screen-document`, what stops your FastAPI worker from crashing with an Out-of-Memory (OOM) error?

• *Judge Focus: Request payload validation, Content-Length limits, and Pillow `Image.MAX\_IMAGE\_PIXELS` decompress bomb protection.*

Q8.2: Can a user bypass the React frontend and directly send forged MRZ JSON to `/api/passengers/new` without a valid document image?

• *Judge Focus: API authentication, CSRF tokens, Pydantic schema validation, and server-side mandatory image verification.*

Q8.3: Are uploaded passenger passport images stored on the server? If so, where and how are they encrypted?

• *Judge Focus: Zero-knowledge architecture, ephemeral memory buffers, and disk cleanup routines.*

ROUND 9: PRIVACY, ETHICS & HUMAN-IN-THE-LOOP

Q9.1: Does your biometric matching process comply with India's Digital Personal Data Protection (DPDP) Act 2023?

• *Judge Focus: Explicit consent, purpose limitation, non-persistence of raw biometric crops, and cryptographically hashed audit trails.*

Q9.2: What is your system's False Rejection Rate (FRR) across different racial demographics and skin tones under varying checkpoint lighting?

• *Judge Focus: Algorithmic bias, illumination sensitivity, and skin-chrominance YCrCb fallback limitations.*

Q9.3: Should an AI system ever have the authority to autonomously detain a human being at a national border?

• *Judge Focus: Human-in-the-loop governance: The AI only flags and recommends; detention requires human officer legal authority.*

ROUND 10: SCALABILITY & AIRPORT PRODUCTION WORKLOAD

Q10.1: Indira Gandhi International Airport (DEL) handles 200,000 passengers daily. How does this architecture scale to handle 10,000 concurrent travelers during peak morning hours?

• *Judge Focus: Replacing Vercel serverless with Kubernetes-managed FastAPI pods, Celery/Redis task queues, and GPU-accelerated CV nodes.*

Q10.2: What is the primary computational bottleneck in your backend pipeline: the FFT liveness check, the ELA re-save, or the OpenCV face cascade?

• *Judge Focus: Profiling knowledge: The JPEG 90% re-save and write-back cycle in memory is the highest CPU consumer.*

ROUND 11: PERFORMANCE & BENCHMARK VALIDATION

Q11.1: You claim '< 1.2 seconds' turnaround. Was this measured on local localhost or across the public internet on Vercel?

• *Judge Focus: Benchmark transparency: Differentiating between algorithmic execution time (< 380ms) and network upload time.*

Q11.2: How many physical document specimens were tested to establish your accuracy and latency claims?

• *Judge Focus: Empirical dataset size, benchmark methodology, and synthetic vs real-world document sample distributions.*

ROUND 12: CORE INNOVATION & COMPETITIVE DIFFERENTIATION

Q12.1: OpenCV, Sobel filters, and ICAO checksums are publicly available algorithms. What is the novel IP created by 'The Deciders'?

• Judge Focus: Explaining the composite correlation engine: cross-validating VIZ against MRZ with multi-spectral forensic fusion in a single-pass pipeline.

Q12.2: Why wouldn't the Bureau of Immigration just buy an off-the-shelf German or French e-Gate system?

• Judge Focus: Cost barrier (\$150,000/gate vs edge-software), indigenization (Make in India), 22 Scheduled Indian Languages, and unified cyber-defense integration.

ROUND 13: VIDEO SCRIPT & DEMO DEFENSE

Q13.1: At 1:10–1:35 in your demo video, you click 'Screen Document' on an altered passport. Was that document processed dynamically in real-time or loaded from a pre-computed sample?

• Judge Focus: Demonstrating live dynamic calculation vs static preset playback.

Q13.2: In the video, the ELA heatmap shows an orange glow around the photo. Exactly which mathematical parameter in `ela.py` caused that color threshold?

• Judge Focus: Connecting UI visual feedback directly to underlying OpenCV color map logic (`cv2.COLORMAP\_INFERNO`).

ROUND 14: CODE DEEP DIVE (FILE-BY-FILE EXAMINATION)

Q14.1: In `backend/main.py`, show me how exceptions raised by `mrz\_parser.py` are caught so the whole server doesn't crash on a corrupted image.

• Judge Focus: Defensive programming: try/except blocks, HTTP 422 vs HTTP 500 error responses.

Q14.2: Explain line-by-line how `backend/forensics/photo\_tampering.py` estimates the boundary box coordinates of the photo without training a YOLO model.

• Judge Focus: Heuristic geometry vs bounding-box regression.

ROUND 15: TEAM MEMBER DEFENSE & DIVISION OF WORK

Q15.1: Which specific files and modules did YOU personally write, and which technical challenges did you solve?

• Judge Focus: Verifying genuine teamwork and weeding out non-contributing team members.

Q15.2: If we delete `src/BorderSuite.jsx`, what happens to the Rakshak Cyber Suite in `src/cyber/`? Are they tightly coupled?

• Judge Focus: Architecture modularity and component isolation.

ROUND 16: RAKSHAK CYBER-DEFENSE SUITE INTEGRATION

Q16.1: Why does a border identity document screening system have an SMS scam and UPI QR code detector inside it?

• Judge Focus: Strategic defense: Explaining holistic border security — stopping physical counterfeiters at the gate while stopping digital cyber-syndicates targeting citizens.

Q16.2: How does your Bloom Filter in `src/cyber/engine/bloomFilter.js` prevent false positives when screening malicious phishing domains?

• Judge Focus: Bloom filter mathematical principles: optimal bit array sizing, hash function counts, and acceptable false positive trade-offs.

ROUND 17: BREAK-THE-SYSTEM ADVERSARIAL STRESS TEST

Q17.1: Scenario: A traveler presents a completely authentic, valid French passport, but they are the identical twin of the passport holder. Does AegisBorder AI grant entry?

• Judge Focus: Admitting biometrics limits: 2D facial geometry cannot differentiate identical twins; requires 3D iris scanning or fingerprint e-Passport chip PKI verification.

Q17.2: Scenario: A passenger has heavy flash glare covering two characters in the MRZ zone. What does your OCR and checksum engine do?

• Judge Focus: Error propagation handling: Fallback to Visual Inspection Zone (VIZ) with an automated flag for manual officer review.

Q17.3: Scenario: The passport was physically laundered/water-damaged, distorting paper noise variance. Will your system falsely classify this legitimate traveler as a criminal?

• Judge Focus: False positive management: The risk engine flags for secondary human inspection rather than immediate hostile detention.